

Performance Optimization Handbook

Doc Ref: ACQO-DOC-002 | v1.0 | 2025

Autonomous Code Quality & Optimization Intelligence System

1. Overview

Performance optimization is the practice of identifying and eliminating inefficiencies in software systems to improve responsiveness, throughput, and resource utilization. This handbook defines optimization strategies applied across all repositories in the ACQO system, from algorithm-level tuning to infrastructure-level caching and query optimization.

2. Current Performance Baseline

Observed service performance metrics (as of Feb 2025) serve as optimization targets:

Service	Avg Response (ms)	Peak Response (ms)	Error Rate (%)	Throughput (req/s)	Status
payment-service	220	780	0.8	320	Monitor
analytics-engine	480	1200	1.2	150	Action Required
auth-service	180	450	0.4	410	Healthy
dashboard-api	260	600	0.6	290	Monitor

■ Action Required: analytics-engine avg response time (480ms) exceeds the 400ms SLO threshold. Peak response at 1200ms indicates blocking I/O or unoptimized pipeline operations.

3. Algorithm Optimization Strategies

Algorithmic inefficiency is the highest-impact category of performance issue. The following strategies must be applied before infrastructure-level solutions are considered:

Issue Pattern	Description	Optimization Approach	Expected Gain
Nested loop over large collections	O(n²) complexity in data processing	Replace with hash map lookup (O(1))	10–100x speedup
Repeated computation in loop	Same value recalculated each iteration	Hoist computation outside loop	5–20x speedup
Recursive without memoization	Exponential call tree for overlapping subproblems	Apply dynamic programming / cache	100x+ speedup
Unnecessary object creation	Allocating objects in hot paths	Reuse objects; use object pooling	2–5x speedup
Blocking synchronous I/O	Waiting on disk/network in main thread	Convert to async/non-blocking I/O	3–10x latency reduction

4. Memory Optimization Techniques

- **Object Pooling:** For high-allocation services like payment-service, pre-allocate and reuse object pools to reduce GC pressure.
- **Lazy Loading:** Defer loading of large datasets until explicitly needed. Critical for analytics-engine pipeline initialisation.
- **Stream Processing:** Replace in-memory collection loading with stream-based processing to cap memory footprint during batch jobs.
- **Cache Eviction Policies:** Use LRU (Least Recently Used) eviction to prevent unbounded cache growth in long-running services.
- **Weak References:** Use weak references for cache entries that should not prevent garbage collection.

5. Database Query Optimization

Anti-Pattern	Problem	Optimized Approach	Priority
SELECT *	Fetches unused columns, bloats I/O	Select only required columns	High
N+1 Query	One query per record in a loop	Use JOIN or batch fetch (IN clause)	High
Missing Index	Full table scan on filtered columns	Add index on WHERE / JOIN columns	High
Unbound Result Set	Fetching millions of rows at once	Use pagination (LIMIT / OFFSET)	Medium
Non-sargable predicates	Functions in WHERE prevent index use	Rewrite to allow index seek	Medium
Cartesian JOIN	Missing join condition produces all pairs	Validate all JOIN conditions explicitly	High

6. Caching Strategies

Caching must be applied at the appropriate layer based on data mutability and access frequency:

Cache Layer	Technology	Best For	TTL Guidance	Invalidation Strategy
In-process (L1)	Caffeine, Guava (Java)	Frequently read config / lookup data	5–60 min	Time-based + manual flush
Distributed (L2)	Redis, Memcached	Session state, API responses	1–24 hr	Key-based + TTL expiry
CDN / Edge	CloudFront, Fastly	Static assets, public API responses	1 hr – 7 days	Cache-Control headers
Database query cache	Query result materialisation	Expensive aggregation queries	15–60 min	Write-through on update

7. Latency Reduction Practices

- **Connection Pooling:** Reuse DB and HTTP connections rather than creating new ones per request. auth-service achieves 180ms avg due to optimised pool configuration.
- **Async Processing:** Offload non-critical operations (email, audit logging) to async queues (e.g. Kafka, SQS) to reduce request path latency.
- **Payload Compression:** Enable gzip/Brotli compression on API responses exceeding 1KB to reduce network transfer time.

- **Parallel Execution:** Use CompletableFuture (Java) or asyncio (Python) to parallelise independent downstream calls.

Performance Optimization Workflow

